Aéza Analyzer

Веб-приложение для анализа риска продления услуг и выручки под риском. Стек: FastAPI, React, PostgreSQL и Docker Compose.

Запуск

Нужны Docker и Docker Compose. Создайте .env, задайте собственный пароль базы данных и запустите приложение:

```
Copy-Item .env.example .env
# Задайте POSTGRES_PASSWORD в .env
docker compose up --build -d
docker compose exec api python -m backend.users create analyst@example.com
```

Интерфейс: http://localhost:8080. API и его справка: http://localhost:8000/api/v1/docs.

Данные и оценки

При первом запуске приложение загружает dataset/data.json: синтетический набор из 10 аккаунтов, 30 услуг и 100 транзакций. Его можно заменить через PUT /api/v1/source/snapshot.

Оценка риска — объяснимая взвешенная эвристика, а не вероятность оттока: в наборе нет размеченных исходов. Выручка под риском считается отдельно по цене ближайшего продления и оценке риска услуги.

Проверка

```
make doctor          # окружение: инструменты, .env, порты (ничего не меняет)
make test            # backend + модуль аутентификации
make openapi-check   # спецификация API совпадает с приложением
make docs-check      # документация, ссылки, спецификации модулей
make verify          # всё перечисленное сразу
```

Без make: `python -m pytest backend/tests -q`, `python -m pytest tests/test_auth.py -q`, `cd frontend && npm ci && npm run build`.

Полная матрица «утверждение → команда → CI»: `docs/verification/matrix.md`.

Документация

Английская версия: `docs/en/index.md`. Каждый документ есть на русском и английском и в двух форматах — `.md` и `.pdf` (`docs/pdf/ru/`, `docs/pdf/en/`); расхождение языков ловит `make docs-check`. Карта модулей для заимствования: `docs/modules/module-map.md`.

Точка входа — **docs/index.md** (карта всех документов).

- Сборка с чистой машины — закреплённые версии, уровни A–E
- Запуск и обновление · Устройство и расчёты
- Схема и миграции · API

- Спецификации модулей · Эксплуатация
- Матрица проверок · Аудит переносимости

Лицензия

MIT — см. LICENSE. Использование, изменение и распространение разрешены при сохранении уведомления об авторских правах.

Управление проектом: `forge.sh` и `forge.bat`

Один скрипт на Unix-подобные системы (`forge.sh`) и один на Windows (`forge.bat`). Интерфейс одинаковый: те же команды, та же логика, одна реализация на каждой платформе — без bash-скриптов внутри Windows.

```
./forge.sh              # интерактивное меню
./forge.sh doctor       # проверить окружение, ничего не меняя
./forge.sh install      # поставить зависимости
./forge.sh start        # запустить в фоне, дождаться готовности, открыть браузер
./forge.sh status       # что запущено и отвечает ли порт
./forge.sh logs api -f  # логи в реальном времени
./forge.sh test         # тесты
./forge.sh stop         # остановить
```

```
forge.bat
forge.bat start
forge.bat status
```

Что умеет

Команда	Действие
`status`	состояние по портам и HTTP, а не «на глаз»
`start [--fg]`	фон (PID-файлы в `.run/`) или передний план
`stop`	останавливает дерево процессов целиком, порт освобождается
`restart`, `logs [имя] [-f]`, `open`	обычные операции
`doctor`	инструменты, версии, `.env`, `.env` в git, занятость порта
`install`, `test`, `lint`, `build`, `verify`	через `make`, если он есть

| `db migrate\|backup\|restore` | база: миграции, копия, восстановление | | `docker up\|down\|logs\|reset` | контейнеры (`down` данные не удаляет) | | `clean [--hard]` | артефакты и логи; `--hard` ещё `.venv` и `node_modules` |

Правила, заложенные в скрипт

- **Ничего не меняет без команды.** `doctor` и `status` только читают.
- **Данные не удаляются.** `stop` и `docker down` сохраняют базу; удаление — только `--hard` или явное подтверждение в `docker reset`.
- **Предполётная проверка.** Если нет `go`/python или занят порт — скрипт говорит об этом сразу, а не через две минуты таймаута.

- **Не висит после падения.** Если процесс умер, запуск прекращается и показывает последние строки лога.
- **Venv главнее системы.** Если есть `.venv`, им пользуются `make`, `pytest` и uvicorn — чтобы `install` и `test` не расходились.

Текущий способ запуска без скрипта: `uvicorn main:app --port 8000` — адрес `http://localhost:8000/api/v1/docs`.

Конфигурация: `.env`

Скрипт читает `.env` **собственным парсером**, а не через `source`: файл приходит из репозитория, а выполнять из него произвольный код нельзя. Поддержаны комментарии `#`, пустые строки, префикс `export`, одинарные и двойные кавычки, CRLF, пробелы вокруг `=` и комментарий после значения.

Приоритет значений: **переменная окружения → `.env` → встроенное значение**. Поэтому `PORT=9000 ./forge.sh start` работает независимо от файла.

Команда	Действие
`env` / `env show`	переменные, источник каждого значения и замаскированные секреты
`env init [--force]`	создать `.env` из `.env.example`: заглушки секретов заменяются случайными, права `600`, прежний файл — в `.env.bak`
`env set KEY=VALUE`	записать или заменить одну переменную (атомарно, через временный файл)
`env check`	сверка с `.env.example`: пропавшие переменные, пустые и «примерные» секреты, `.env` в `.gitignore`
`env edit`	открыть файл в `$EDITOR` (fallback: nano → vim → vi)
`env path`, `env reload`	путь к файлу и перечитывание без перезапуска

Секреты в выводе маскируются: `ADMIN_CODE=ch…de (31 симв.)`, пароль внутри строки подключения — `postgres://user:***@host`. Имена вида `*_MINUTES`, `*_URL`, `*_PORT` секретами не считаются: маскировать число минут бессмысленно.

Путь к файлу переопределяется: `FORGE_ENV_FILE=/etc/app.env ./forge.sh status`. В Windows-версии те же команды: `forge.bat env init`, `forge.bat env check`. Ограничение `cmd.exe`: значения, содержащие `%` и `!`, не поддерживаются.

Флаги и автодополнение

```
./forge.sh -h | --help            # справка
./forge.sh -V | --version         # версия скрипта, коммит, состояние .env
./forge.sh -q status              # молчать об успехах
./forge.sh --no-color status      # без ANSI (то же делает NO_COLOR=1)
./forge.sh --json status          # машинный вывод для скриптов и мониторинга

eval "$(./forge.sh completion bash)"   # автодополнение для bash
./forge.sh completion zsh > ~/.zfunc/_forge   # для zsh
```

`test`, `lint`, `build` и `verify` запускаются в окружении **без** переменных из `.env`: иначе локальные проверки вели бы себя иначе, чем CI, и тесты вида «без ключа должен быть fallback» перестали бы что-либо проверять.

Запуск в этом проекте

```
./forge.sh install && ./forge.sh start
```